

Rapport technique

Assistant intelligent de recommandation
d'événements culturels

Projet 7
Parcours AI Engineer
OpenClassrooms

Auteur : Viken KHATCHERIAN

Technologies : Python, LangChain, FAISS, Mistral, FastAPI

Architecture : Retrieval Augmented Generation (RAG)

Table des matières

1	Objectifs du projet	3
1.1	Contexte	3
1.2	Problématique	3
1.3	Objectif du POC	3
1.4	Périmètre	4
2	Architecture du système	5
2.1	Vue globale de l'architecture	5
2.2	Diagramme UML de l'architecture	5
2.3	Flux d'interrogation utilisateur	7
2.4	Technologies utilisées	7
2.4.1	Langage	7
2.4.2	Frameworks et bibliothèques	7
2.4.3	Modèles NLP utilisés	7
2.5	Conclusion architecture	8
3	Préparation et vectorisation des données	9
3.1	Source des données	9
3.2	Nettoyage et normalisation NLP	9
3.3	Construction des documents RAG	10
3.4	Chunking sémantique	10
3.5	Génération des embeddings	11
4	Construction de la base vectorielle	12
4.1	Utilisation de FAISS	12
4.2	Stratégie de persistance	12
4.3	Métadonnées conservées	12
4.4	Recherche documentaire	13
5	Choix du modèle NLP	14
5.1	Modèle de génération sélectionné	14
5.2	Justification du choix	14
5.3	Construction du prompt	14
5.4	Limites du modèle	15
6	API et endpoints exposés	16
6.1	Framework utilisé	16
6.2	Endpoints disponibles	16
6.2.1	Vérification du service	16

6.2.2	Recherche d'événements	16
6.2.3	Reconstruction de l'index	17
6.3	Documentation Swagger	17
6.4	Format des requêtes et réponses de l'API	17
6.5	Gestion des erreurs	18
7	Évaluation du système RAG	19
7.1	Jeu de test	19
7.2	Métriques utilisées	19
7.3	Résultats obtenus	19
7.4	Analyse quantitative des résultats	20
7.5	Analyse qualitative	20
8	Recommandations et perspectives	22
8.1	Ce qui fonctionne bien	22
8.2	Limites du POC	22
8.3	Améliorations possibles	22
9	Organisation du dépôt GitHub	23
10	Annexes	24
10.1	Extrait du jeu de test annoté	24
10.2	Exemple de réponse JSON générée par l'API du RAG	25
10.3	Exemple de log d'évaluation	26
10.4	Conclusion	27

Chapitre 1

Objectifs du projet

1.1 Contexte

Dans le cadre du projet Puls-Events, l'objectif est de concevoir un assistant intelligent capable d'aider les utilisateurs à trouver des événements culturels adaptés à leurs besoins.

Les utilisateurs doivent pouvoir poser des questions en langage naturel comme :

- activité enfant à Paris ;
- événement culturel à Versailles ;
- festival de musique ;
- sortie famille dans les Hauts-de-Seine.

La difficulté principale consiste à exploiter un volume important de données événementielles tout en fournissant des réponses pertinentes et compréhensibles.

1.2 Problématique

Une recherche classique par mots-clés présente plusieurs limites :

- absence de compréhension du sens d'une question ;
- difficulté à gérer les synonymes ;
- résultats peu adaptés au contexte utilisateur.

Une architecture Retrieval Augmented Generation (RAG) permet de combiner :

- une recherche sémantique dans une base vectorielle ;
- un modèle de langage capable de générer une réponse naturelle.

Le système ne répond donc pas uniquement avec la mémoire du modèle : il utilise des documents issus des données OpenAgenda.

1.3 Objectif du POC

Ce Proof Of Concept (POC) vise à démontrer :

- la faisabilité technique d'un assistant RAG ;
- la capacité à retrouver les événements pertinents ;
- la réduction des hallucinations du modèle ;
- l'exposition du système via une API REST.

1.4 Périmètre

Les données utilisées proviennent de l'API publique OpenAgenda. Un compte a été créé sur <https://openagenda.com/fr> pour obtenir une clé d'accès aux endpoints :

- "https://api.openagenda.com/v2/agendas" ;
- "https://api.openagenda.com/v2/agendas/uid/events"

Le périmètre couvre :

- événements culturels ;
- activités famille et enfants ;
- événements localisés sur l'ensemble des départements d'Ile-de-France.

Les réponses sont générées à partir d'une base documentaire construite à partir des événements collectés.

Chapitre 2

Architecture du système

2.1 Vue globale de l’architecture

L’architecture proposée repose sur une approche Retrieval-Augmented Generation (RAG). Elle sépare la phase de préparation des données de la phase d’interrogation utilisateur.

Le système est composé de plusieurs couches :

- une couche d’acquisition permettant de récupérer les événements depuis l’API publique OpenAgenda ;
- une couche de préparation des données réalisant le nettoyage, la normalisation et la transformation des événements en documents exploitables par un système RAG ;
- une couche de recherche sémantique basée sur des embeddings multilingues et un index vectoriel FAISS ;
- une chaîne RAG utilisant LangChain pour orchestrer la recherche, le reranking, la construction du prompt et la génération ;
- une couche d’exposition via une API REST FastAPI conteneurisée avec Docker.

2.2 Diagramme UML de l’architecture

Le diagramme suivant représente les principaux composants du système et les flux de données entre eux.

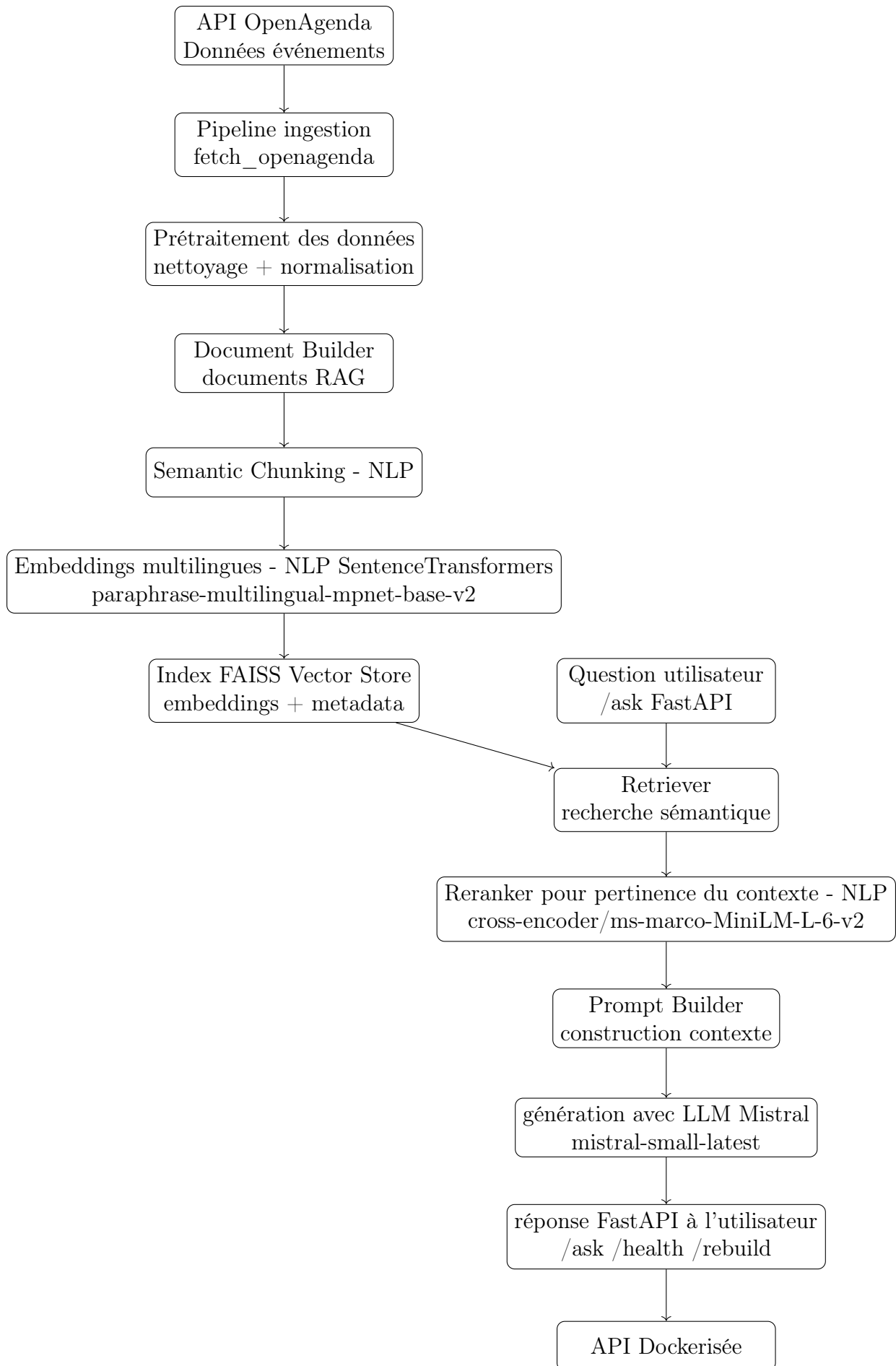


FIGURE 2.1 – Architecture globale du système RAG

2.3 Flux d'interrogation utilisateur

Lorsqu'un utilisateur pose une question, le flux est le suivant :

1. La question utilisateur est reçue par l'API FastAPI via l'endpoint `/ask`.
2. La question est transformée en représentation vectorielle grâce au modèle SentenceTransformers.
3. Le Retriever interroge l'index FAISS afin de récupérer les documents les plus proches sémantiquement.
4. Le Reranker améliore le classement des documents récupérés grâce à un Cross Encoder.
5. Le Prompt Builder construit un contexte enrichi contenant la question et les informations pertinentes.
6. Le modèle Mistral génère une réponse naturelle.
7. L'API retourne la réponse JSON au client.

Remarque : les étapes retriever, reranker, prompt builder et réponse générée par le LLM mistral du flux sont orchestrées par le script `src/rag/rag_chain_v3.py`. Ce script d'orchestration est également utilisé par l'API exposant les services du RAG.

2.4 Technologies utilisées

2.4.1 Langage

Le projet est développé en :
— Python 3.11

2.4.2 Frameworks et bibliothèques

Les principales technologies utilisées sont :

- **LangChain** : utilisé pour construire la chaîne RAG et orchestrer les différentes étapes.
- **SentenceTransformers** : utilisé pour produire les représentations vectorielles.
- **HuggingFace Transformers** : utilisé pour les modèles NLP.
- **FAISS** : utilisé comme base vectorielle.
- **FastAPI** : utilisé pour exposer les fonctionnalités via une API REST.
- **Docker** : utilisé pour la conteneurisation.

2.4.3 Modèles NLP utilisés

Le traitement automatique du langage naturel intervient à plusieurs niveaux.

- **Embeddings sémantiques**

Le modèle :

`paraphrase-multilingual-mpnet-base-v2`

de SentenceTransformers permet de transformer les documents et les questions en vecteurs multilingues.

- **Recherche sémantique**
Le même modèle est utilisé afin de comparer les similarités entre la question utilisateur et les événements disponibles.
- **Reranking**
Le modèle :
`cross-encoder/ms-marco-MiniLM-L-6-v2`
est utilisé afin de réordonner les documents retournés par FAISS.
- **Génération**
La génération finale est réalisée par :
`Mistral mistral-small-latest`
via l'API Mistral et LangChain.

2.5 Conclusion architecture

Cette architecture permet d'obtenir un assistant capable de rechercher des événements pertinents tout en limitant les réponses inventées grâce à une génération contrôlée par un contexte documentaire issu de FAISS.

Chapitre 3

Préparation et vectorisation des données

3.1 Source des données

Les données utilisées dans ce projet proviennent de l'API publique **OpenAgenda**. Cette source fournit des informations sur des événements culturels et de loisirs :

- titre de l'événement ;
- description ;
- catégorie ;
- localisation ;
- date et horaires ;
- informations complémentaires.

La collecte est réalisée automatiquement par le script :

```
src/ingestion/fetch_openagenda_v34.py
```

Les données récupérées sont ensuite stockées sous forme de fichiers JSON afin d'être utilisées dans les étapes suivantes du pipeline RAG.

3.2 Nettoyage et normalisation NLP

Une étape de nettoyage est appliquée afin d'obtenir des documents homogènes et exploitables par les modèles NLP.

Le traitement est réalisé par :

```
src/preprocessing/clean_events_v2.py
```

Les opérations réalisées sont notamment :

- suppression des espaces inutiles ;
- nettoyage des champs texte vides ;
- normalisation des descriptions ;
- suppression des caractères parasites ;
- harmonisation des métadonnées ;
- vérification de la présence des informations nécessaires.

Cette étape permet d'améliorer la qualité des embeddings générés ensuite.

3.3 Construction des documents RAG

Après nettoyage, les événements sont transformés en documents adaptés à une architecture Retrieval-Augmented Generation.

Cette étape est réalisée avec :

```
src/preprocessing/document_builder_v2.py
```

Chaque événement est converti sous la forme :

- un contenu textuel exploitable par le LLM ;
- des métadonnées associées permettant le filtrage ;
- un identifiant unique.

Exemple de structure obtenue :

```
{
  "page_content": "Atelier jeux enfants à Paris ...",

  "metadata": {
    "title": "Atelier jeux enfants",
    "city": "Paris",
    "category": "famille"
  }
}
```

3.4 Chunking sémantique

Les documents sont ensuite découpés en morceaux appelés *chunks*.

Le découpage est réalisé avec :

```
RecursiveCharacterTextSplitter
```

de la bibliothèque LangChain.

L'objectif est de conserver des passages suffisamment riches pour la recherche sémantique tout en respectant les contraintes des modèles d'embeddings.

Le script utilisé est :

```
src/preprocessing/semantic_chunk_events_v5.py
```

Chaque chunk conserve :

- son texte ;
- son identifiant ;
- les métadonnées de l'événement d'origine.

3.5 Génération des embeddings

La représentation vectorielle des documents est réalisée avec SentenceTransformers.
Le modèle utilisé est :

`paraphrase-multilingual-mpnet-base-v2`

Ce modèle permet :

- la compréhension multilingue ;
- la comparaison entre questions et documents ;
- une recherche basée sur le sens et non uniquement sur les mots.

La génération est réalisée par :

`src/embeddings/embed_semantic_events_v4.py`

Chaque document devient un vecteur numérique.

Le format obtenu correspond à :

- un tableau de nombres flottants ;
- une dimension fixe définie par le modèle ;
- une correspondance un vecteur = un chunk documentaire.

Chapitre 4

Construction de la base vectorielle

4.1 Utilisation de FAISS

Le stockage des embeddings est réalisé avec **FAISS (Facebook AI Similarity Search)**. FAISS permet une recherche rapide des vecteurs les plus proches selon une distance de similarité.

La construction de l'index est réalisée avec :

```
src/vectorstore/faiss_semantic_index_v5.py
```

Le pipeline réalise :

1. chargement des embeddings ;
2. création de l'index FAISS ;
3. association entre vecteurs et documents ;
4. sauvegarde persistante.

4.2 Stratégie de persistance

Afin de conserver la base vectorielle entre deux exécutions, l'index FAISS est sauvegardé sur disque.

Les éléments persistés sont :

- l'index FAISS contenant les vecteurs ;
- les métadonnées associées ;
- les documents utilisés pour la reconstruction.

La persistance permet :

- d'éviter de recalculer les embeddings ;
- de démarrer rapidement l'API ;
- de reconstruire la base à la demande via l'endpoint `/rebuild`.

L'index généré est sauvegardé dans le fichier nommé `semantic_events.index` et les métadonnées sont enregistrées dans le fichier nommé `semantic_events_metadata.json`.

4.3 Métadonnées conservées

Chaque document conserve des informations permettant d'expliquer et filtrer les résultats.

Les principales métadonnées sont :

- titre de l'événement ;
- description ;
- catégorie ;
- ville ;
- date ;
- identifiant du chunk ;
- source OpenAgenda.

Ces métadonnées sont également utilisées lors de la génération afin que le LLM puisse produire une réponse structurée contenant les informations utiles.

4.4 Recherche documentaire

Lors d'une question utilisateur, le Retriever interroge FAISS.

Le fonctionnement est :

1. transformation de la question en embedding ;
2. recherche des vecteurs les plus proches ;
3. récupération des documents associés ;
4. transmission au Reranker.

Cette étape constitue la première partie de la chaîne RAG.

Chapitre 5

Choix du modèle NLP

5.1 Modèle de génération sélectionné

La génération des réponses est réalisée avec le modèle de langage **Mistral**.
Le modèle utilisé est :

```
mistral-small-latest
```

L'accès au modèle est réalisé via l'API Mistral et intégré dans la chaîne RAG avec LangChain.

Le composant responsable est :

```
src/rag/mistral_generator_v3.py
```

5.2 Justification du choix

Le choix de Mistral repose sur plusieurs critères :

- bonne qualité de génération en français ;
- compatibilité avec LangChain ;
- temps de réponse adapté à un POC ;
- coût maîtrisé grâce à une API distante ;
- capacité à suivre un contexte fourni par un système RAG.

5.3 Construction du prompt

Le prompt est construit automatiquement avant l'appel au modèle.
Cette étape est réalisée par :

```
src/rag/prompt_builder_v3.py
```

Le prompt contient :

- la question utilisateur ;
- le contexte récupéré par FAISS ;
- les consignes de génération ;
- les règles de limitation des hallucinations.

Le modèle reçoit uniquement les informations issues des documents récupérés.

Cette approche permet d'obtenir des réponses basées sur les données OpenAgenda.

5.4 Limites du modèle

Les limites identifiées sont :

- dépendance à la qualité du contexte récupéré ;
- risque de reformulation incorrecte ;
- absence de mémoire conversationnelle ;
- coût lié aux appels API.

Chapitre 6

API et endpoints exposés

6.1 Framework utilisé

L'application est exposée sous forme d'une API REST développée avec **FastAPI**.
Le point d'entrée est :

`api/main.py`

6.2 Endpoints disponibles

6.2.1 Vérification du service

Endpoint :

GET /health

Réponse :

```
{  
  "status": "ok",  
  "service": "rag-api"  
}
```

6.2.2 Recherche d'événements

Endpoint :

POST /ask

Exemple :

```
{  
  "question": "activité enfant à Paris"  
}
```

Le système :

1. transforme la question en embedding ;
2. recherche dans FAISS ;
3. applique le reranking ;
4. construit le prompt ;
5. appelle Mistral ;
6. retourne la réponse.

6.2.3 Reconstruction de l'index

Endpoint :

POST /rebuild

Cette route permet de reconstruire automatiquement :

- les documents RAG ;
- les embeddings ;
- l'index FAISS.

6.3 Documentation Swagger

FastAPI fournit automatiquement une documentation interactive.

Adresse :

<http://localhost:8000/docs>

Elle permet :

- de tester les endpoints ;
- de visualiser les modèles de requêtes ;
- d'exécuter des appels API.

6.4 Format des requêtes et réponses de l'API

L'API exposée via Swagger utilise un format d'échange basé sur des objets JSON. Les requêtes envoyées au service doivent respecter le format suivant :

```
{
  "question": "string"
}
```

Le champ `question` correspond à la demande formulée par l'utilisateur. La réponse retournée par l'API contient la question reçue ainsi que la réponse générée, selon le format suivant :

```
{
  "question": "string",
  "answer": "string"
}
```

Par exemple, pour une requête demandant des événements pour enfants à Paris, l'API peut retourner une réponse contenant les informations associées aux événements trouvés :

```
{
  "question": "événements pour enfants à Paris ?",
  "answer": "1. Parcours d'équilibre
  - Catégorie : loisir
  - Ville : Paris
  - Date : 16 mars 2026 à 10h00
  - Description : Les enfants vont passer le parcours moteur
    pour appréhender l'équilibre."
}
```

6.5 Gestion des erreurs

L'API gère notamment les questions vides.

Une question hors domaine retourne une réponse contrôlée :

"Je ne sais pas"

afin de limiter les hallucinations.

Dans le cas où le champ `question` est vide, l'API retourne une erreur indiquant que la question est obligatoire :

```
{
  "detail": "Question cannot be empty"
}
```

Chapitre 7

Évaluation du système RAG

7.1 Jeu de test

L'évaluation utilise un dataset annoté contenant :

30 questions utilisateur

Chaque question possède :

- une réponse attendue ;
- des événements attendus ;
- des mots-clés ;
- des éléments interdits.

7.2 Métriques utilisées

L'évaluation combine plusieurs critères :

- similarité sémantique entre réponse générée et référence ;
- couverture des informations attendues ;
- détection d'hallucination ;
- classification finale : correct, partial ou incorrect.

7.3 Résultats obtenus

Résultats après amélioration du pipeline :

Statut	Nombre
Correct	22
Partiel	6
Incorrect	2

TABLE 7.1 – Résultats de l'évaluation RAG

Accuracy obtenue :

73,3 %

7.4 Analyse quantitative des résultats

Une évaluation quantitative du système a été réalisée sur un ensemble de 30 questions afin de mesurer la qualité des réponses générées. Plusieurs indicateurs ont été pris en compte : le score obtenu par le modèle, la similarité entre la réponse générée et la réponse attendue, la présence des informations obligatoires ainsi que le taux d'hallucination.

Sur les 30 réponses analysées, 22 réponses ont été classées comme *correctes*, 6 comme *partielles* et 2 comme *incorrectes*. Le taux de réponses correctes atteint ainsi environ 73%, ce qui montre que le système est capable de répondre efficacement à la majorité des requêtes. Les réponses partielles représentent 20% des tests et correspondent principalement à des cas où les informations retournées sont présentes mais incomplètes ou moins précises par rapport aux attentes.

L'analyse des scores montre une bonne performance générale avec plusieurs résultats supérieurs à 0.8. Les meilleures performances sont observées sur les tests 1, 2 et 12 avec des scores respectifs de 0.951, 0.926 et 0.908, indiquant une forte correspondance entre la question et la réponse générée. La similarité atteint également des valeurs satisfaisantes, avec un maximum de 0.805 pour le premier test, ce qui indique une bonne capacité du système à retrouver des informations pertinentes.

Les réponses classées comme partielles présentent généralement des scores plus faibles, notamment les tests 13, 22 et 3 avec des scores respectifs de 0.553, 0.546 et 0.575. Ces résultats montrent que le système identifie une partie du besoin utilisateur mais peut manquer certains éléments attendus. Les deux cas incorrects correspondent aux tests 5 et 30, avec des scores faibles de 0.156 et 0.245. Ces erreurs sont principalement liées à des questions trop générales ou à une absence de correspondance suffisante avec les données disponibles.

Enfin, aucun cas d'hallucination n'a été détecté sur les 30 requêtes évaluées. L'ensemble des réponses générées reste donc basé sur les informations disponibles dans la base de connaissances. Cette absence d'informations inventées confirme que le système privilégie la pertinence des données récupérées plutôt que la génération de contenu non vérifié.

Globalement, l'évaluation quantitative montre que le système fournit des réponses fiables dans la majorité des situations testées, avec un bon niveau de précision, une capacité correcte de récupération d'informations et une bonne maîtrise du risque d'hallucination.

7.5 Analyse qualitative

Exemple de réponse correcte :

Question :
activité enfant à Paris

Réponse :
Atelier jeux enfants à partir de 2 mois
Ville : Paris
Catégorie : famille

Exemple de limitation :

Une question hors domaine comme :

vol sur la planète Mars

n'est pas couverte par les données OpenAgenda.

Le système répond qu'il ne possède pas l'information.

Chapitre 8

Recommandations et perspectives

8.1 Ce qui fonctionne bien

Le système fonctionne correctement pour :

- recherche culturelle ;
- activités familiales ;
- recherche par localisation ;
- questions en langage naturel.

Le pipeline RAG permet de réduire fortement les hallucinations grâce à la génération basée sur contexte.

8.2 Limites du POC

Les limites principales sont :

- dépendance aux données OpenAgenda ;
- volume limité d'événements ;
- absence de mémoire conversationnelle ;
- coût des appels API Mistral ;
- temps de reconstruction FAISS.

8.3 Améliorations possibles

Les évolutions envisagées :

- ajout d'une recherche hybride BM25 + embeddings ;
- amélioration du reranking ;
- ajout d'une mémoire conversationnelle ;
- synchronisation automatique OpenAgenda ;
- monitoring des performances ;
- déploiement cloud.

Chapitre 9

Organisation du dépôt GitHub

Organisation principale :

```
api/  
    API FastAPI  
  
src/  
    ingestion  
    preprocessing  
    embeddings  
    vectorstore  
    rag  
  
data/  
    test_dataset  
  
tests/  
    tests unitaires  
    tests intégration  
  
Dockerfile  
requirements.txt  
README.md
```

Rôle de chaque répertoire :

- api pour héberger les scripts schemas.py, routes.py et main.py qui lancent l'API ;
- src pour les scripts d'ingestion, de préparation, de vectorisation et d'indexation des données ainsi que les scripts de construction et d'évaluation de la chaîne RAG ;
- data pour héberger le jeu d'annotations de test, les réponses attendues, les réponses générées et le rapport d'évaluation ;
- tests pour héberger les scripts de tests unitaires et d'intégration de la chaîne RAG ainsi que de l'API ;

Remarque : le Dockerfile et les dépendances requirements.txt pour la conteneurisation de l'API du RAG sont à la racine du projet.

Chapitre 10

Annexes

10.1 Extrait du jeu de test annoté

Afin d'évaluer les performances du système de recherche et de génération de réponses, un jeu de test annoté a été construit. Chaque entrée du jeu de données contient un identifiant unique ainsi qu'une question formulée en langage naturel correspondant à une demande utilisateur réelle. Ces questions couvrent plusieurs domaines liés aux événements familiaux, aux activités pour enfants et aux sorties culturelles en Île-de-France.

Un extrait du jeu de test utilisé pour l'évaluation est présenté ci-dessous :

```
[
  {
    "id": 1,
    "question": "activité enfant à Paris"
  },
  {
    "id": 2,
    "question": "atelier parent enfant dans les Hauts-de-Seine"
  },
  {
    "id": 3,
    "question": "activité petite enfance en Île-de-France"
  },
  {
    "id": 4,
    "question": "sortie famille près de Versailles"
  },
  {
    "id": 5,
    "question": "événement famille à Saint-Denis"
  },
  {
    "id": 6,
    "question": "événement culturel à Paris"
  },
  {
    "id": 7,
```

```

    "question": "spectacle ou activité culturelle en Seine-Saint-Denis"
  },
  {
    "id": 8,
    "question": "exposition à Paris"
  },
  {
    "id": 9,
    "question": "activité artistique dans le Val-de-Marne"
  }
]

```

Chaque question de ce jeu de test est ensuite utilisée comme entrée du système. Les réponses générées sont comparées aux résultats attendus à l'aide des métriques d'évaluation précédemment définies, notamment le score global, la similarité cosinus entre les représentations vectorielles des réponses, la présence des informations obligatoires et la détection d'hallucinations. Ce jeu de données permet ainsi de mesurer la robustesse du système sur différents types de requêtes et différents contextes géographiques.

10.2 Exemple de réponse JSON générée par l'API du RAG

Un exemple de test réalisé via l'interface Swagger consiste à rechercher des activités destinées aux enfants à Paris. La requête envoyée au service est la suivante :

```

{
  "question": "activités enfants à Paris ?"
}

```

La réponse retournée par l'API contient une liste d'événements correspondant aux critères demandés. Chaque événement est présenté avec son titre, sa catégorie, sa localisation, sa date et une description :

```

{
  "answer": "1. Titre : La Semaine Nationale De La Petite Enfance
  Catégorie : loisir
  Ville : Paris
  Date : 16 mars 2026
  Description : Organisation d'activités autour du thème
  Des Équilibres.

  2. Titre : atelier parents enfants pro
  Catégorie : famille
  Ville : Paris
  Date : 16 mars 2026
  Description : Lors de la semaine de la petite enfance,
  des ateliers parents-enfants et professionnels sont proposés."
}

```

```

3. Titre : lieu accueil enfants/parents
Catégorie : famille
Ville : Paris
Date : 19 mars 2026
Description : Espaces conviviaux et ludiques pour favoriser
l'éveil et le jeu des enfants de 0 à 6 ans accompagnés
d'un parent.

4. Titre : Atelier jeux enfants à partir de 2 mois
Catégorie : famille
Ville : Paris
Date : 17 mars 2026
Description : Espace d'échanges parents/professionnels de PMI
sur le thème des jeux pour enfants à partir de 2 mois."
}

```

Cet exemple montre que le système est capable d'identifier les informations importantes présentes dans la question utilisateur et de retourner des résultats structurés contenant les détails des événements correspondants.

10.3 Exemple de log d'évaluation

Afin d'évaluer automatiquement les performances du système, un mécanisme de journalisation (*logging*) a été mis en place. Pour chaque question testée, le système génère un enregistrement contenant plusieurs métriques permettant d'analyser la qualité de la réponse produite. Ces métriques permettent notamment d'évaluer la pertinence sémantique de la réponse, sa conformité avec les informations attendues et la fiabilité du contenu généré.

Un exemple de log obtenu lors de l'évaluation est présenté ci-dessous :

```

{
  "id": 2,
  "score": 0.926,
  "similarity": 0.704,
  "required": 1,
  "hallucination": 0,
  "status": "correct"
}

```

Dans cet exemple, le test identifié par l'ID 2 obtient un score global de 0.926. Cette valeur représente une mesure synthétique de la qualité de la réponse générée en prenant en compte plusieurs critères d'évaluation. Une valeur élevée indique que la réponse respecte les attentes définies lors du test.

La métrique **similarity** correspond à la similarité cosinus calculée entre l'embedding de la réponse générée et celui de la réponse de référence. Cette mesure permet de comparer la proximité sémantique entre deux textes, même lorsque les formulations utilisées sont différentes. Elle est comprise entre 0 et 1 : une valeur proche de 1 indique que les deux réponses portent un sens très proche, tandis qu'une valeur faible indique une divergence

importante. Dans ce cas, une similarité de 0.704 montre que la réponse générée possède une bonne correspondance avec la réponse attendue.

Le champ **required** permet de vérifier la présence des informations obligatoires dans la réponse. Une valeur égale à 1 signifie que les éléments nécessaires ont bien été retrouvés et intégrés dans la réponse finale. À l'inverse, une valeur égale à 0 indique qu'au moins une information attendue est absente.

Le champ **hallucination** permet de contrôler la fiabilité du modèle en détectant la génération d'informations non présentes dans les données disponibles. Une valeur de 0 indique qu'aucune hallucination n'a été détectée, ce qui signifie que la réponse reste basée sur les informations récupérées. Une valeur de 1 signalerait au contraire la présence d'informations inventées ou non vérifiées.

Enfin, le champ **status** représente la classification finale du résultat. Dans cet exemple, le statut *correct* confirme que la réponse satisfait les critères définis : score élevé, similarité suffisante avec la référence, présence des informations requises et absence d'hallucination.

10.4 Conclusion

Ce projet démontre la faisabilité d'un assistant intelligent de recommandation culturelle basé sur une architecture RAG.

L'association de :

- NLP ;
- embeddings multilingues ;
- FAISS ;
- LangChain ;
- reranking ;
- LLM Mistral ;
- API FastAPI ;
- Docker ;

permet d'obtenir un prototype fonctionnel répondant aux objectifs du POC.

Auteur : Viken KHATCHERIAN